# Project 2: Medical Data Pipeline with Knowledge Graphs

Sergi Flores, Paula Esteve, Claudia Gallego

June 3, 2026

## 1 Introduction

The goal of this project is to continue building a complete end-to-end data science pipeline by extending the previous work with a knowledge graph layer and graph-based analysis. The new scope focuses on how the RDF representation is designed, how it is queried, and how graph embeddings can support downstream modeling.

Due to the significant size of the generated data files and Spark environments, the full source code, configuration files, and implementation details are hosted on GitHub. You can access the complete repository at the following link: `https://github.com/gallego-c/bda-practica2`.

## 2 Previous work

In previous project, we cfreated a 5 stage pipeline integrating 3 different datasets. They were from the heathcare domain, focusing on cardiovascular diseases. The first dataset, *Cardiovascular Heart Diseases*, The *Heart Disease Health Indicators* from BRFSS and he *Cleveland Heart Disease dataset*. Our goal was to evaluate the risk of developing such diseases based on both medical and behavioral factors.

The architecture as mentioned was divided in 5 stages. The first Zone was the Landing Zone. It ingests three configured Kaggle datasets and stores each extraction as a timestamped raw Parquet snapshot. Keeping ingestion separate from downstream transformations and serves as a stable reference point.

The second was the Formatting Zone, that standardizes the data to make sure that the three source datasets are consistent and structured in the same way. Spark is used and the datasets are aligned at the schema level. Then, we use DuckDB to store the formatted data for easy access and a lightweight storage solution.

The Trusted Zone is the third layer of the data pipeline, where transformations are applied, strict data-quality constraints and rules are enforced on the standardized data. This zone ensures that the data meets necessary quality standards before moving to the next stages.

In the Exploitation Zone, trusted data is integrated and processed to create the combined dataset used in analysis. Any discrepancies between datasets are resolved, and the data is transformed into the unified structure. The data is then aggregated into a singular dataset, the canonical table, thanks to a vertical union.

Finally, the Analysis Zone trains two reproducible *integrated* machine learning pipelines over the canonical table. Both analytical pipelines follow the same execution template with some difference in the models.

## 3 Project Overview

This project keeps the original five-zone data engineering structure but adds a semantic layer on top of the integrated health data. The main contribution is graph model that represents records, indicators, population groups, and outcomes that is suitable for analysis and reusable querying.

Additionally two graph-oriented tasks: SPARQL exploration of the generated RDF and KGE-based analysis over graph-derived representations. Both are meant to demonstrate how the same integrated dataset can support relational and semantic analytics.

## 4 Project improvements

Based on the feedback received in the first project, we refined the pipeline to better separate the responsibilities of each zone. In the Formatted Zone, we removed data preparation and cleaning tasks, limiting this stage to schema standardization and the generation of structured SQL tables. Data-quality checks and cleaning operations were moved to the Trusted Zone, where they belong.

We also simplified the Exploitation Zone by avoiding unnecessary intermediate storage: instead of saving the data as Parquet and reading it again, Spark now processes the trusted data directly when building the exploitation outputs. Finally, we redesigned the Analysis Zone taking advantage of the fact that it was one of the premises of this new project.

## 5 Knowledge graphs

The knowledge graph was built to represent the integrated cardiovascular data in a semantic model that is more expressive than a flat table. In the explotation zone, each trusted record becomes a HealthRecord instance and the shared conceptual layers are represented through PopulationGroup, AgeGroup, Gender, Indicator, and Outcome classes.

Object properties such as belongsToAgeGroup, hasGender, belongsToPopulationGroup, hasObservedRiskFactor, and hasOutcome connect records to their context, while data properties store metadata such as run identifiers, aggregate values, and observation counts. This allows as to store all relevant information in one graph schema.

Table 1 summarizes the central schema components, while Table 2 shows the kind of relations that connect the diferent classes.

| **Element** | **Role in the graph** |
|---|---|
| HealthRecord | Trusted row instance |
| Dataset | Source provenance node |
| PopulationGroup | Shared demographic bridge |
| Indicator | Common health concept |
| Outcome | Target concept |
| AggregateMeasurement | Population summary node |

Table 1: Main RDF schema components

At the instance level, a row is materialized as a record node linked to its dataset, age band, gender, population group, outcome, and observed risk or protective factors. In parallel, aggregate nodes (PopulationGroup, AggregateMeasurement) summarize each gender, age and general indicator combinations, which gives the graph both global and local analytical value.

| Node | Connected via |
|---|---|
| HealthRecord | fromDataset, belongsToPopulationGroup, hasObservedRiskFactor |
| PopulationGroup | populationGroupAgeGroup, populationGroupGender |
| AggregateMeasurement | measuresIndicator, forPopulationGroup |
| Indicator | indicatorColumn, indicatorCategory, valueKind |

Table 2: Example graph connections used in the KG

The finnal generated knowledge graph contains more than 3.6 million triples and connects 322,587 health record resources with datasets, indicators, population groups and outcome information.

## 6 Querying with SPARQL

SPARQL is used to interrogate the graph at the semantic level, applying pattern-matching queries over RDF triples to detect recurring structures, compare population groups, and extract aggregate analytical signals from the integrated health data.

In this layer, the goal goes beyond simple retrieval. We transforms raw RDF triples into patterns that are used as evidence about how risk behaves across datasets and population segments. Table 3 summarizes the main pattern families used in the pipeline and the type of analytical output they produce.

| Query objective | Main graph pattern | Analytical output |
|---|---|---|
| Match records for a specific demographic profile | HealthRecord → PopulationGroup → AgeGroup / Gender | Record-level subsets filtered by demographic context |
| Compare indicator prevalence across population groups | AggregateMeasurement → PopulationGroup → Indicator | Relative prevalence patterns across age and gender groups |
| Measure outcome variation by age group | AggregateMeasurement → PopulationGroup → AgeGroup | Ranked outcome rates by demographic segment |
| Detect co-occurring risk factors in positive cases | HealthRecord → hasObservedRiskFactor → Indicator | Co-occurrence counts and frequent pattern signals |

Table 3: Representative SPARQL pattern-matching queries

Instead of querying only exact values, the graph allows us to query structural relationships: which entities are connected, which patterns recur across datasets, and which demographic contexts are associated with higher risk. An example SPARQL query is shown in Listing 1.

```
PREFIX hr: <https://example.org/bda/health-risk/>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

SELECT ?indicator1Label ?indicator2Label
       (COUNT(DISTINCT ?record) AS ?coOccurringRecords)
WHERE {
  ?record a hr:HealthRecord ;
          hr:hasObservedRiskFactor ?indicator1 ;
          hr:hasObservedRiskFactor ?indicator2 ;
          hr:hasOutcome <https://example.org/bda/health-risk/outcome/heart-disease-
              positive> .

  ?indicator1 rdfs:label ?indicator1Label .
  ?indicator2 rdfs:label ?indicator2Label .

  FILTER(STR(?indicator1) < STR(?indicator2))
}
GROUP BY ?indicator1Label ?indicator2Label
ORDER BY DESC(?coOccurringRecords)
LIMIT 20
```

Listing 1: SPARQL query for co-occurring risk factors

The results of this specific query are..

# 7 KGE-based analysis

The KGE-based analysis uses the knowledge graph as a source of structured features. Graph entities and relations are transformed into embeddings, and those embeddings are then used as inputs for predictive models.

This stage turns semantic structure into numerical vectors that can support downstream prediction. The important point is that the graph captures context that is not available in the tabular form alone, so the embedding-based analysis can exploit relationships between population groups and indicators while being compatible with standard machine learning workflows.

The KG Embedding ML pipeline first loads the analytics RDF graph produced in Part 4. To avoid data leakage, outcome measurement nodes are held out before training the embeddings. The pipeline then learns node representations using PyKEEN knowledge graph embedding models, with TruncatedSVD used as a fallback when needed. These embeddings build the record-level feature matrix that is used to train the models. Finally, downstream classifiers are trained and evaluated using these KG-derived features, producing the final models and metrics.

The KG Embedding ML pipeline was compared against the two previous pipelines, created in project 1. They are the Integrated Core pipeline that uses the main aligned variables shared across the datasets and the Integrated Enriched pipeline that adds a broader set of variables.

The objective of the model comparison was not only to find the highest score, but also to understand what type of representation provides more useful information. The core model works as a simpler baseline. The enriched model tests whether adding more harmonised tabular variables improves prediction. The embedding model tests whether the semantic structure of the graph can also support predictive learning.

The models were evaluated using several metrics because accuracy alone can be misleading in this context. The datasets have different class distributions, and the CDC dataset in particular

has a lower positive target rate. For that reason, precision, recall, F1-score, ROC-AUC and PR-AUC were also considered. This gives a more complete view of the behaviour of each pipeline.

| Pipeline | Accuracy | Precision | Recall | F1-score | ROC-AUC |
|---|---|---|---|---|---|
| Integrated Core | 0.7934 | 0.4435 | 0.5869 | 0.5053 | 0.8175 |
| Integrated Enriched | 0.8400 | 0.5493 | 0.6122 | 0.5790 | 0.8693 |
| KG Embedding ML | 0.8211 | 0.5015 | 0.6981 | 0.5837 | 0.8606 |

Table 4: Comparison of the best models obtained in each analytical pipeline.

As shown in Table 4 and Figure 1, the KG Embedding ML pipeline achieved the best F1-score and the highest recall among the three approaches. This is important because recall measures how many positive cases are correctly detected. In health-related prediction tasks, this can be especially relevant, since missing positive cases may be more problematic than producing some false positives.

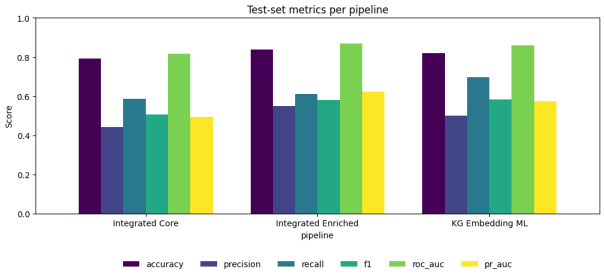


Figure 1: Comparison of the test-set metrics obtained by the three modelling pipelines.

However, the Integrated Enriched pipeline obtained the highest accuracy, precision, and ROC-AUC, which suggests that adding more harmonised information improves the predictive quality of the integrated dataset.

The Integrated Core model also performed reasonably well, which indicates that the common aligned variables already contain relevant predictive information. However, its lower F1-score and precision show that the reduced feature set is not enough to capture all relevant patterns.

Overall, the three models showed strong performance, highlighting the usefulness of the different techniques. The comparison also shows that the knowledge graph is not only useful for SPARQL queries, as its embeddings capture information that can be used by machine learning models.

# 8 Discussion

This project delivers a complete end-to-end data pipeline for cardiovascular risk analysis. The use of separate data zones helped organise the process from raw ingestion to final analysis. Each zone has a clear responsibility, which improves traceability and makes the system easier to validate.

The main difference from the previous work was the integration of the knowledge graph and its two analysis pipelines. The first pipeline used the knowledge graph and SPARQL queries to identify repeated patterns associated with higher risk, while the second pipeline used the knowledge graph for predictive modelling through graph embeddings. The results showed that this approach achieved the highest recall and F1-score values.

The final system is not only able to clean and integrate heterogeneous cardiovascular risk datasets, but also to support both interpretable graph-based analysis and predictive modelling.